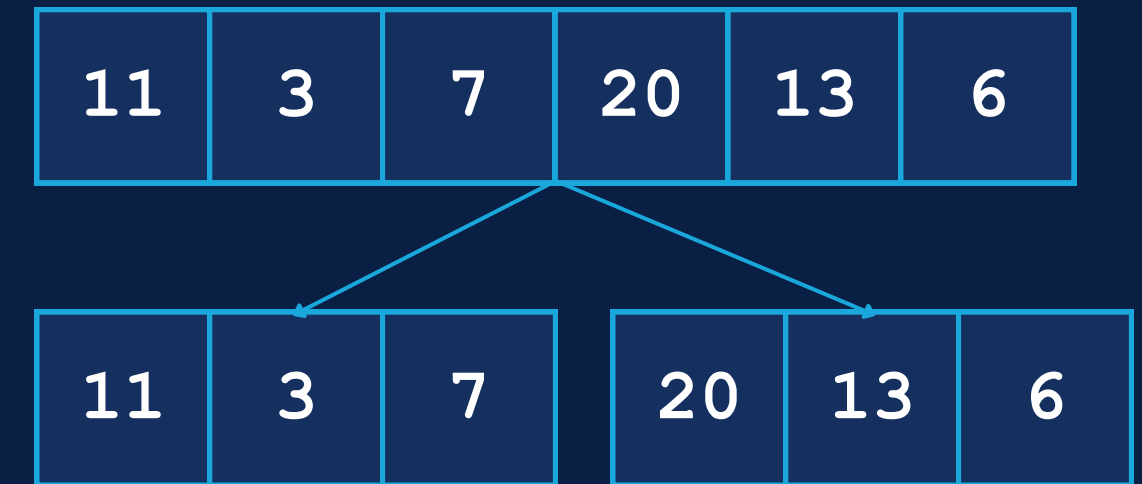


Merge Sort: Implementation, Analysis, and Performance Comparison

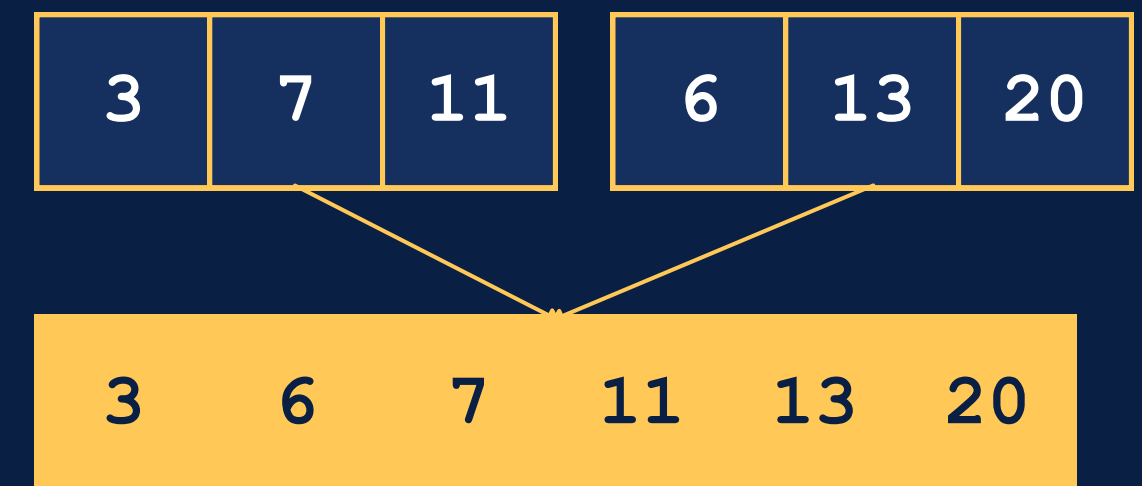
Instructor: Visethboti Sin
COSC 251 · Fall 2026
American University of Phnom Penh

Group members:
Choup Vannphyreak
Taing Tuvatnak
Sarou Menglong
Doung Piphop
Rin Sovandara

DIVIDE



MERGE



Why Sorting Matters

Many programs need their data in order before they can use it

Sorting means arranging data in a defined order, for example from smallest to largest.



Searching

Sorted data is much faster to search



Analysing

Patterns and rankings are easier to find



Organizing

Records stay in a predictable order



Displaying

Users see data in a clear order

Our group's algorithm: Merge Sort

11	3	7	20	13	6
----	---	---	----	----	---

Unsorted array



Sorting algorithm



3	6	7	11	13	20
---	---	---	----	----	----

Sorted array

Project Objectives

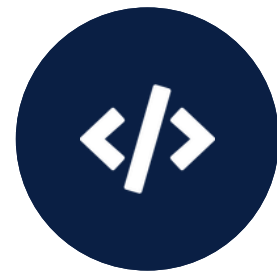
Five goals guided our research, code, and experiments



01

Understand

How Merge Sort works



02

Implement Merge Sort

Written manually in Java



03

Implement 3 baselines

Bubble, Selection, and Insertion Sort



04

Test 3 dataset sizes

5, 1,000, and 1,000,000 elements



05

Compare

Runtime, comparisons, movements/swaps, and complexity

Understand → Implement → Test → Compare

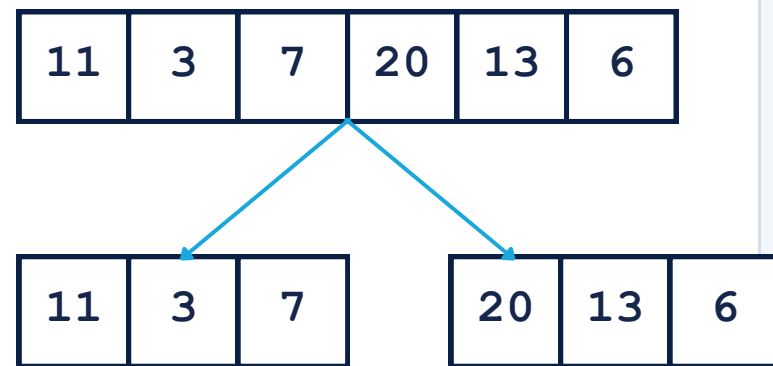
What Is Merge Sort?

A divide-and-conquer sorting algorithm

Split the array into halves, sort the pieces, then merge them back together in order.

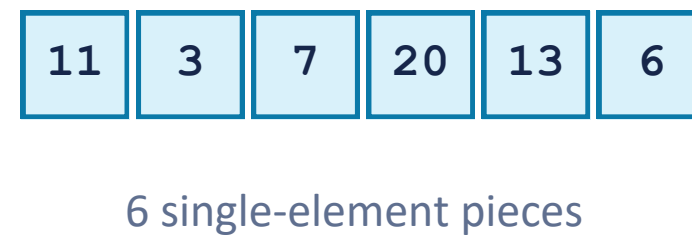
1 Divide

Split the array into two smaller halves



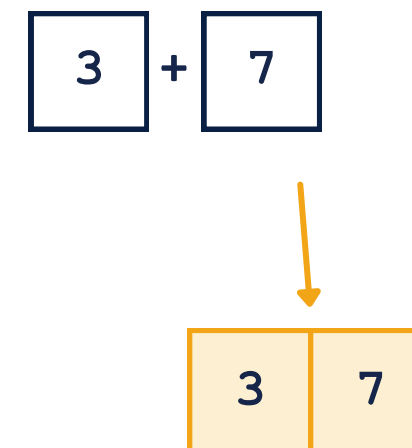
2 Repeat

Keep splitting until each piece has one element



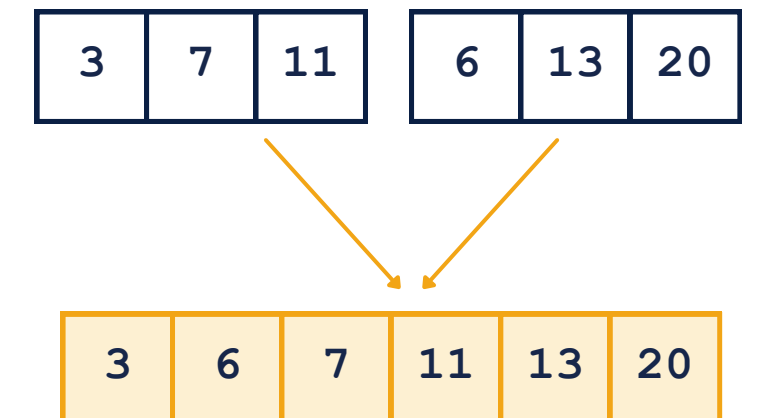
3 Merge

Combine small sorted pieces back together



4 Order

While merging, the smaller front element goes first



Key Characteristics of Merge Sort

What makes Merge Sort different from the basic sorts



Divide and conquer

Split, solve, merge



Recursive

sort() calls itself on each half



Can be stable

Equal values keep their original order — ties take the left element (But our code use <)



Predictable

$O(n \log n)$ in best, average, and worst case



Extra memory

Our array-based version creates temporary arrays: $O(n)$



Good for large data

The gap over $O(n^2)$ sorts grows as n grows



Overhead on tiny arrays

Recursion and extra arrays cost more than they save

The Divide-and-Conquer Idea

Three ideas: divide, conquer, then combine

1

DIVIDE

Split the problem into smaller subproblems.

2

CONQUER

Solve the smaller problems recursively.

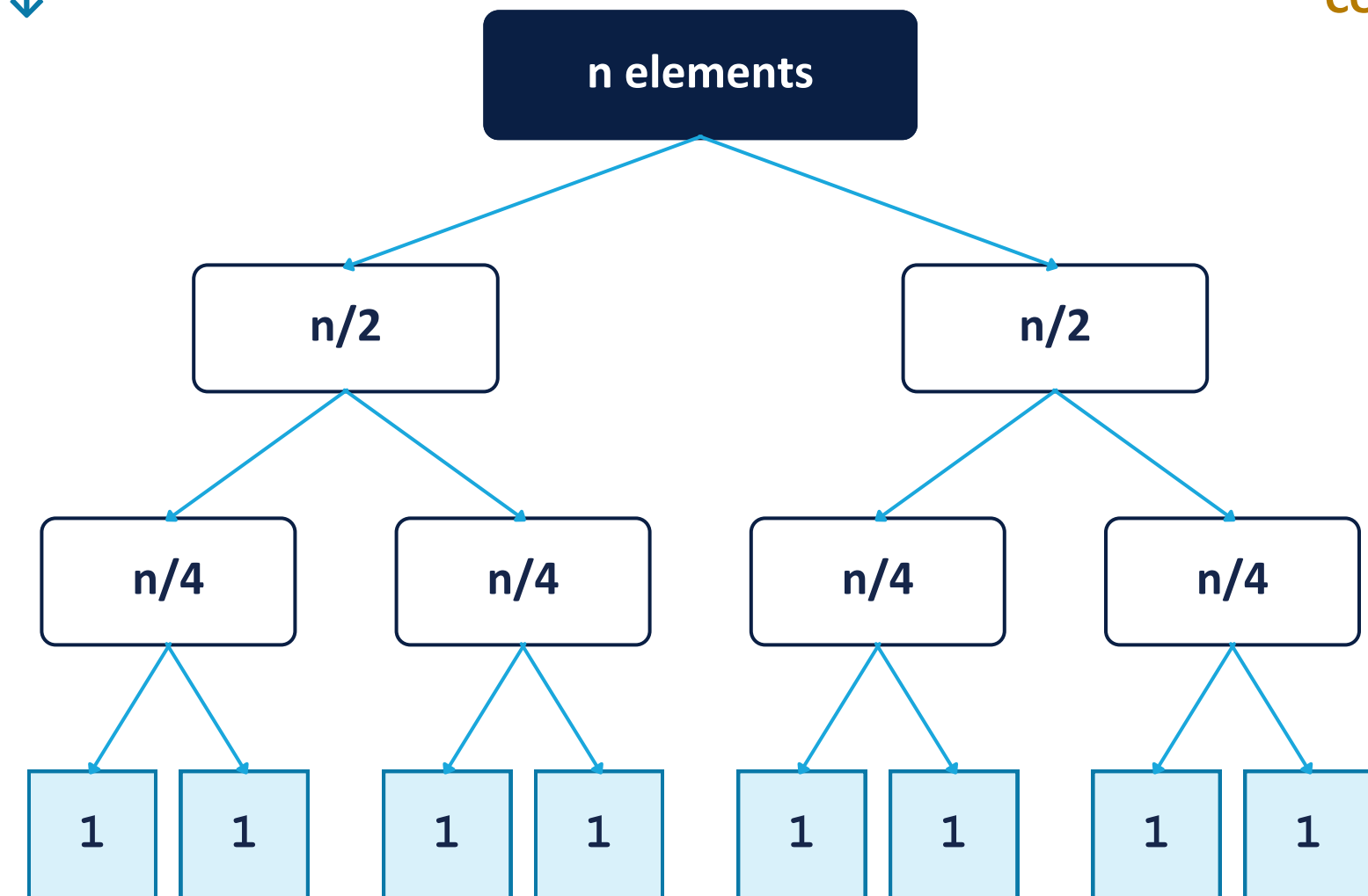
3

COMBINE / MERGE

Combine the solved pieces into one sorted result.

DIVIDE ↓

COMBINE ↑



Splitting stops at single elements. About $\log n$ levels of splitting.

Step 1: Divide Phase

Example array from our report: [11, 3, 7, 20, 13, 6]

Start



Split 1



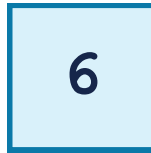
Split 2



Split 3



Result:



Six single-element pieces. A single element is already sorted, so splitting stops.

Step 2: Merge Phase

Sorted pieces are combined, two at a time

LEFT HALF



RIGHT HALF



How merging works



Elements are compared while two sorted pieces are combined: the smaller front element is placed into the result first. Each merged piece is therefore already sorted.

Step 3: Final Merge

Left [3, 7, 11] and right [6, 13, 20] become one sorted array

LEFT

3	7	11
---	---	----

RIGHT

6	13	20
---	----	----

Step

Compare front elements

Result so far

1	3	vs	6	take 3	3
2	7	vs	6	take 6	3 6
3	7	vs	13	take 7	3 6 7
4	11	vs	13	take 11	3 6 7 11
5	Left is empty: copy 13, 20				3 6 7 11 13 20

This merge

4

comparisons

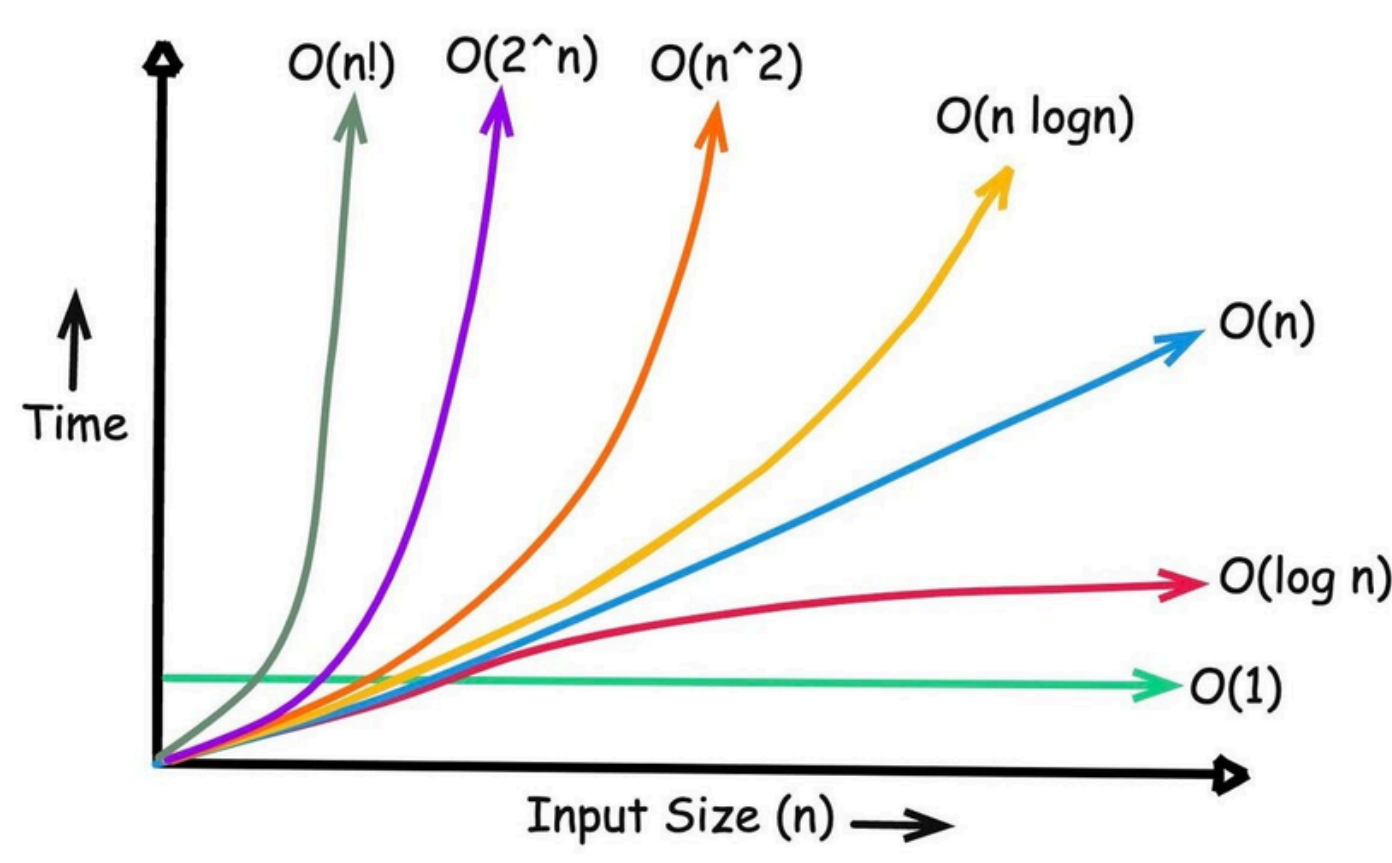
6

movements

Time and Space Complexity

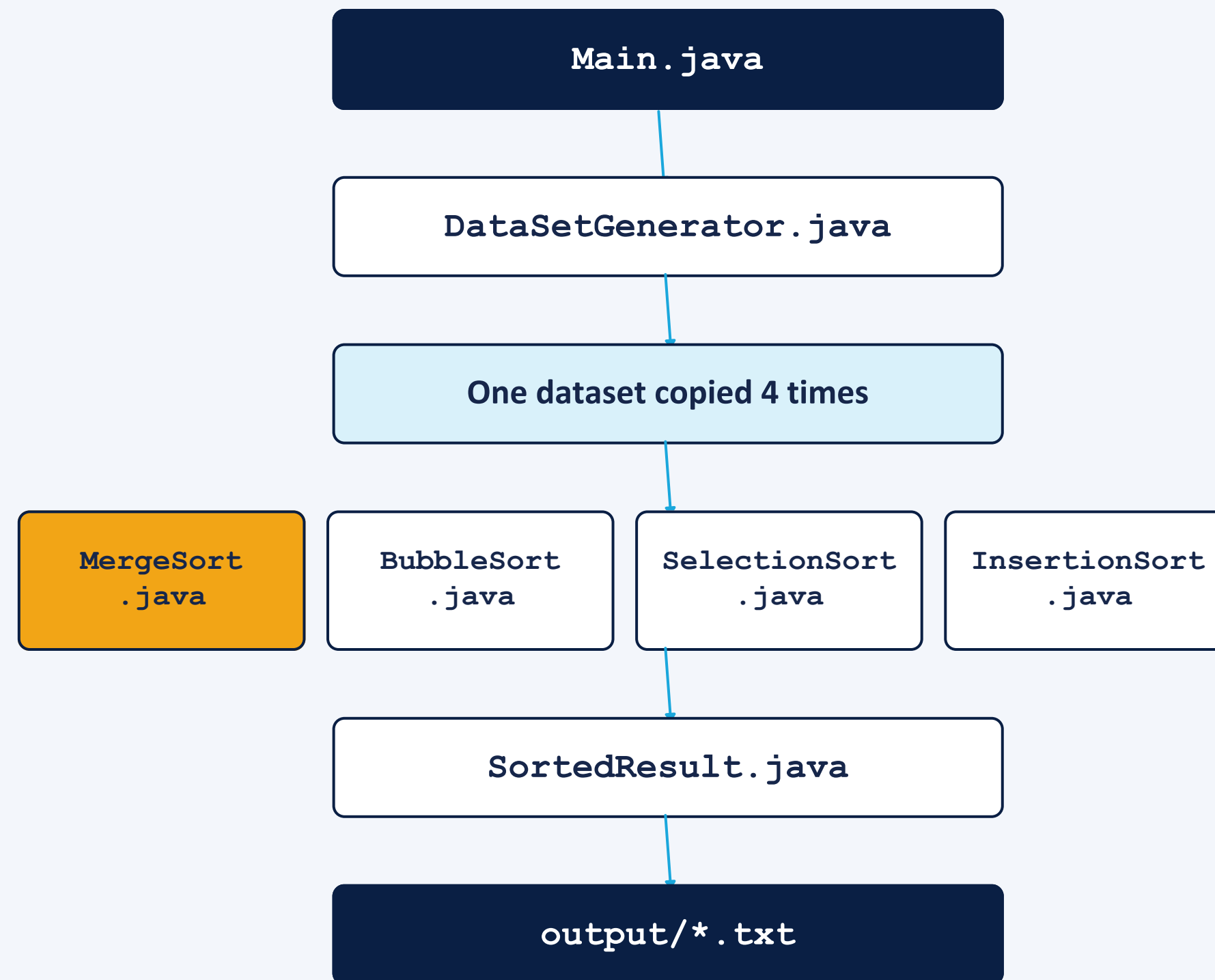
The divide-and-merge structure never changes, so the time does not either

Case	Merge Sort	Why
Best case	$O(n \log n)$	Still divides and merges an already sorted array
Average case	$O(n \log n)$	About $\log n$ levels, and each level merges n elements
Worst case	$O(n \log n)$	Same structure regardless of input order
Space	$O(n)$	Temporary arrays hold the two halves



Our Java Implementation

How the project is organized, from one dataset to saved results



MergeSort.java

The Merge Sort implementation: `sorting()`, `sort()`, and `merge()`

DataSetGenerator.java

Random integers from 0 to 999,999 with a fixed seed, so every algorithm gets the same data

SortedResult.java

Stores comparisons, swaps/movements, and runtime (ns); saves them and the sorted array to .txt

Main.java

Runs all four algorithms on sizes 5, 1,000, and 1,000,000

Core Merge Sort Methods

```
private static void sort(int[] arr, SortedResult result) {
    int arrayLength = arr.length;

    // base case
    if (arrayLength <= 1) {
        return; // return back once the array has only a single value
    }

    // split the array
    int midIndex = arrayLength / 2;
    // left array
    int[] leftArray = new int[midIndex];
    // right array
    int[] rightArray = new int[arrayLength - midIndex];

    int i = 0; // i is for left array index
    int j = 0; // j is for right array index

    // populate the left and right array
    for (; i < arrayLength; i++) {
        // add half of the array in leftArray
        if (i < midIndex) {
            leftArray[i] = arr[i];
        }
        // add the other half in rightArray
        else {
            rightArray[j] = arr[i];
            j++;
        }
    }
    // recursively call the method to keep splitting the array in half
    sort(leftArray, result);
    sort(rightArray, result);

    // call a helper method to merge the array back after comparison
    merge(leftArray, rightArray, arr, result);
}
```

sort()

- Recursively divides the array
- Stops when the array has one element or fewer
- Calls merge() after both halves are sorted

```
// helper method for merging the array back
private static void merge(int[] leftArray, int[] rightArray, int[] array, SortedResult result) {
    // initialise the size of each left and right array
    int leftSize = leftArray.length;
    int rightSize = rightArray.length;

    int i = 0; // initial pointer of the "array"
    int left = 0; // initial index of the "leftArray"
    int right = 0; // initial index of the "rightArray"

    // start comparing both the left and right array together
    while (left < leftSize && right < rightSize) {
        // count the comparison
        result.comparison++;
        if (leftArray[left] < rightArray[right]) {
            array[i] = leftArray[left];
            left++;
        }
        else {
            array[i] = rightArray[right];
            right++;
        }
        i++;
        // count the swap
        result.swap++;
    }

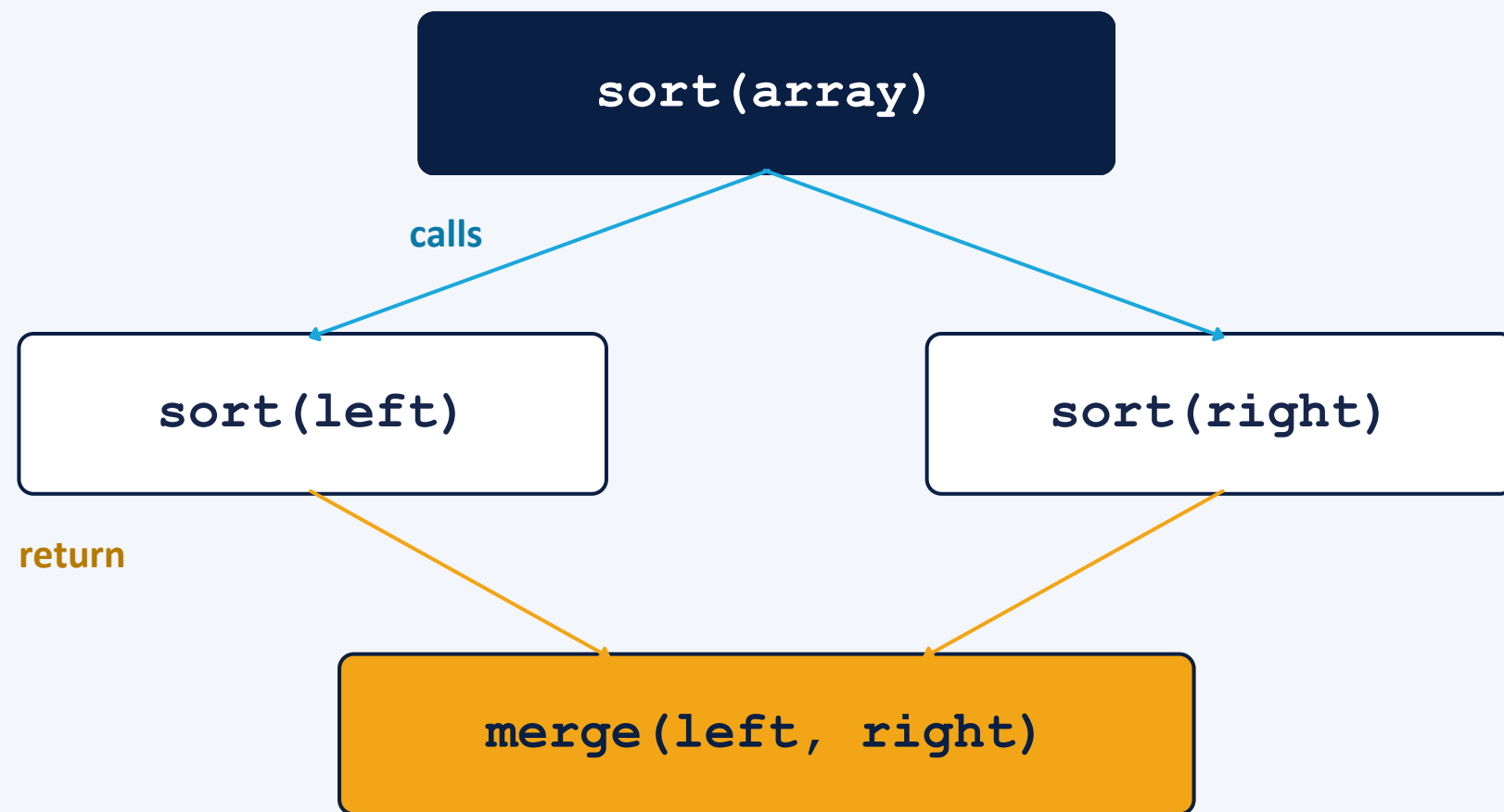
    // add the remaining left or right array into the "array"
    while (left < leftSize) {
        array[i] = leftArray[left];
        i++;
        left++;
        // count the swap
        result.swap++;
    }
    while (right < rightSize) {
        array[i] = rightArray[right];
        i++;
        right++;
        // count the swap
        result.swap++;
    }
}
```

merge()

- Compares the front element of each half
- Places the smaller value into the output array
- Counts comparisons and movements
- Copies any leftover elements at the end

Recursion in Merge Sort

Calls go down until the base case, then merging happens on the way back up



BASE CASE

array length ≤ 1

→ stop recursion (one element is already sorted)

Example: sorting [11, 3, 7]

- 1** `sort([11, 3, 7])`
splits into [11] and [3, 7]
- 2** `sort([11])`
base case, returns
- 3** `sort([3, 7])`
splits into [3] and [7]; both return
- 4** `merge([3], [7])`
gives [3, 7]
- 5** `merge([11], [3, 7])`
gives [3, 7, 11]

Testing Methodology

Every algorithm sorts exactly the same starting data

- 1 Generate one random dataset for each required size
- 2 Use a fixed seed (seed = 5), values 0 to 999,999
- 3 Clone the dataset once for each algorithm
- 4 Give all four algorithms identical starting data
- 5 Record comparisons, swaps/movements, and runtime (ns)
- 6 Save sorted results and metrics to .txt files

DATASET SIZES

Small	5 elements
Medium	1,000 elements
Large	1,000,000 elements

METRICS

Runtime

Comparisons

Swaps / Movements

Identical datasets = a fair comparison

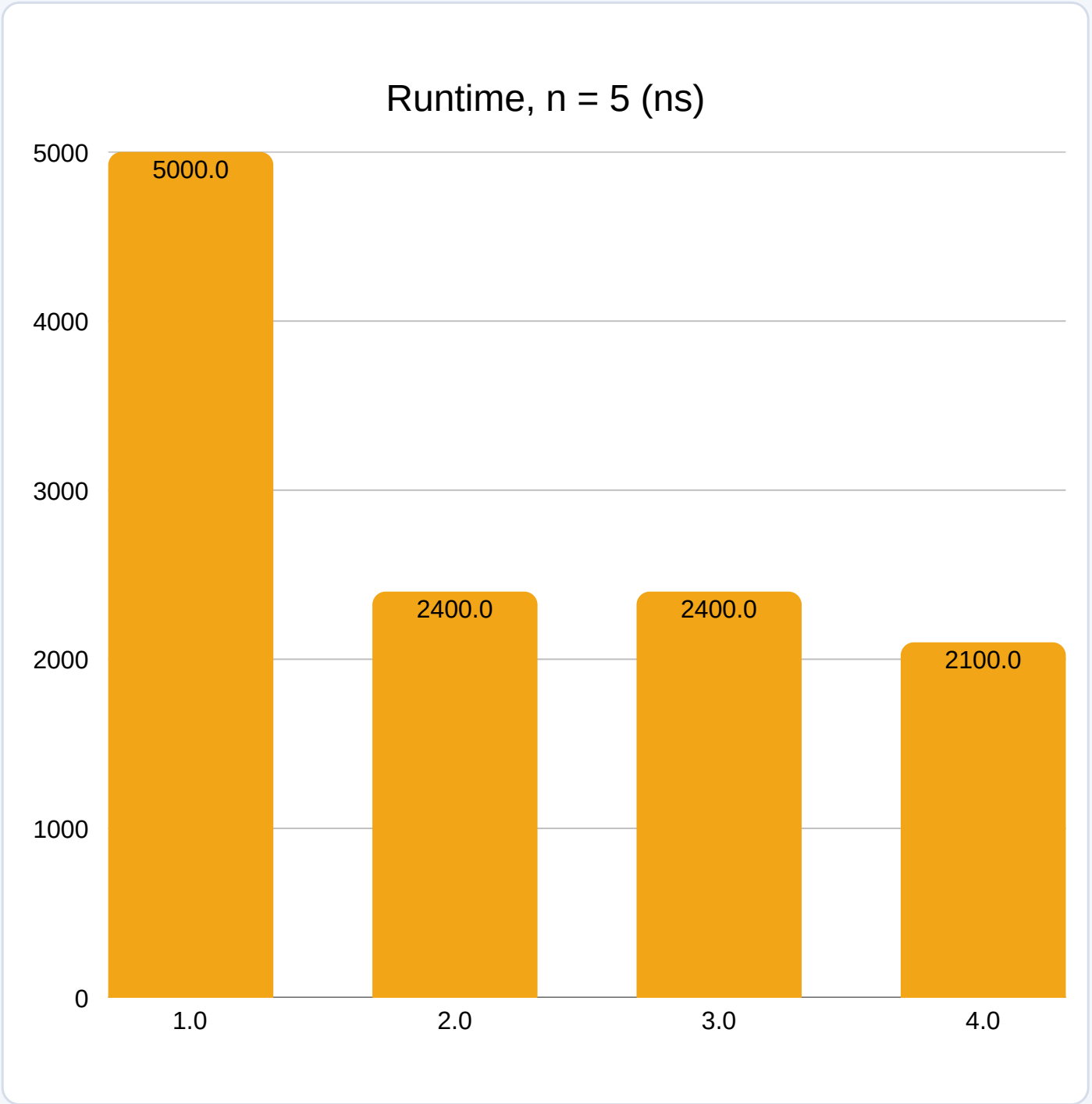
Small Dataset Results (n = 5)

On tiny inputs, simple algorithms can win

Algorithm	Comparisons	Swaps / Movements	Runtime (ns)
Merge Sort	6	12	5000
Bubble Sort	7	2	2,400
Selection Sort	10	2	2,400
Insertion Sort	2	2	2,100

Observation

Merge Sort has the highest runtime here. Recursion and creating temporary arrays add overhead that is not worth it for only 5 elements.



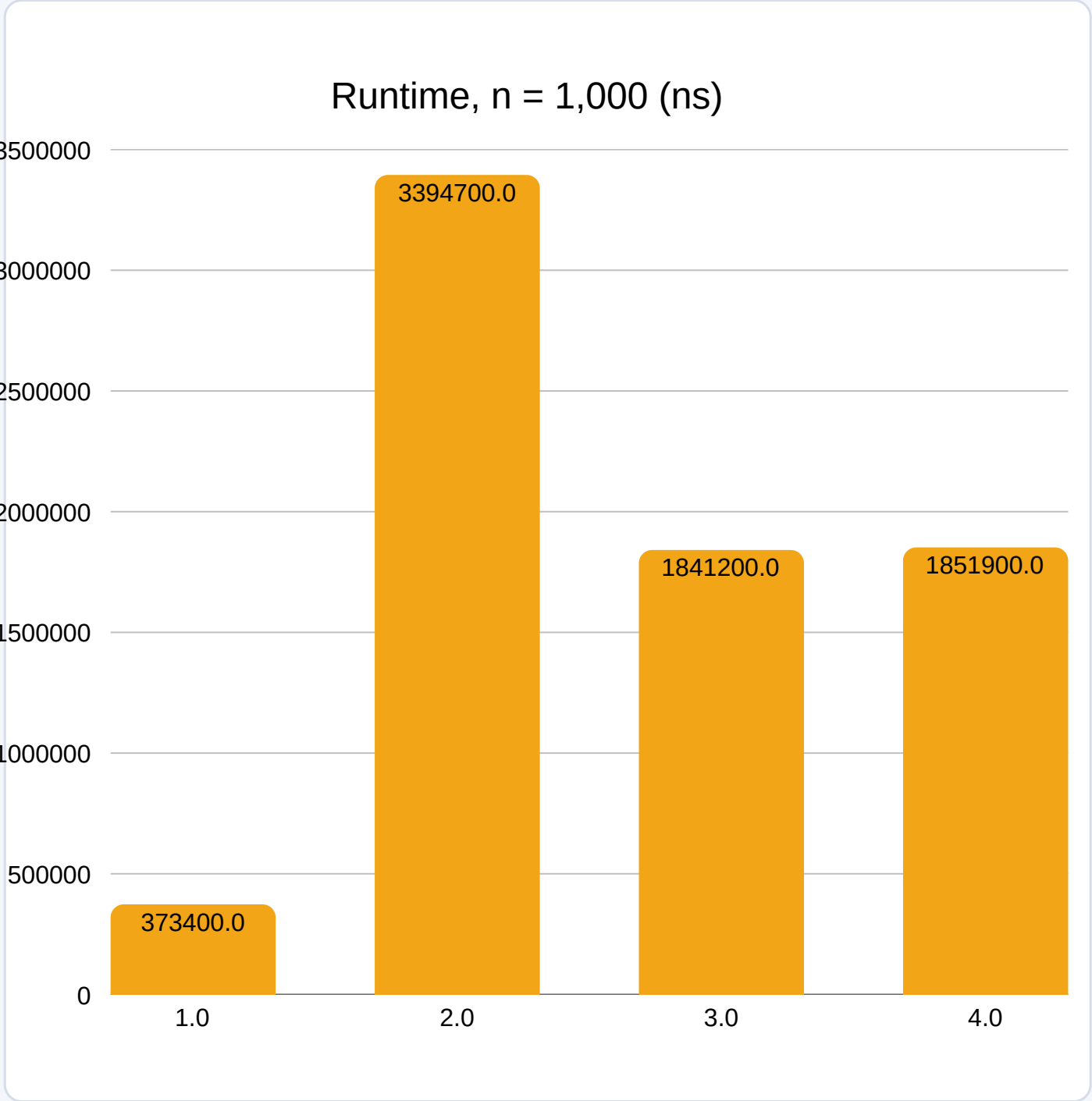
Medium Dataset Results (n = 1,000)

Now the divide-and-conquer structure starts to pay off

Algorithm	Comparisons	Swaps / Movements	Runtime (ns)
Merge Sort	8,690	9,976	373,400
Bubble Sort	497,960	242,099	3,394,700
Selection Sort	499,500	995	1,841,200
Insertion Sort	242,099	242,099	1,851,900

Observation

Merge Sort is fastest: about 5–9× faster than the other three, with about 28–57× fewer comparisons. Selection Sort makes far fewer movements (995).



Large Dataset Results (n = 1,000,000)

The gap becomes dramatic at one million elements

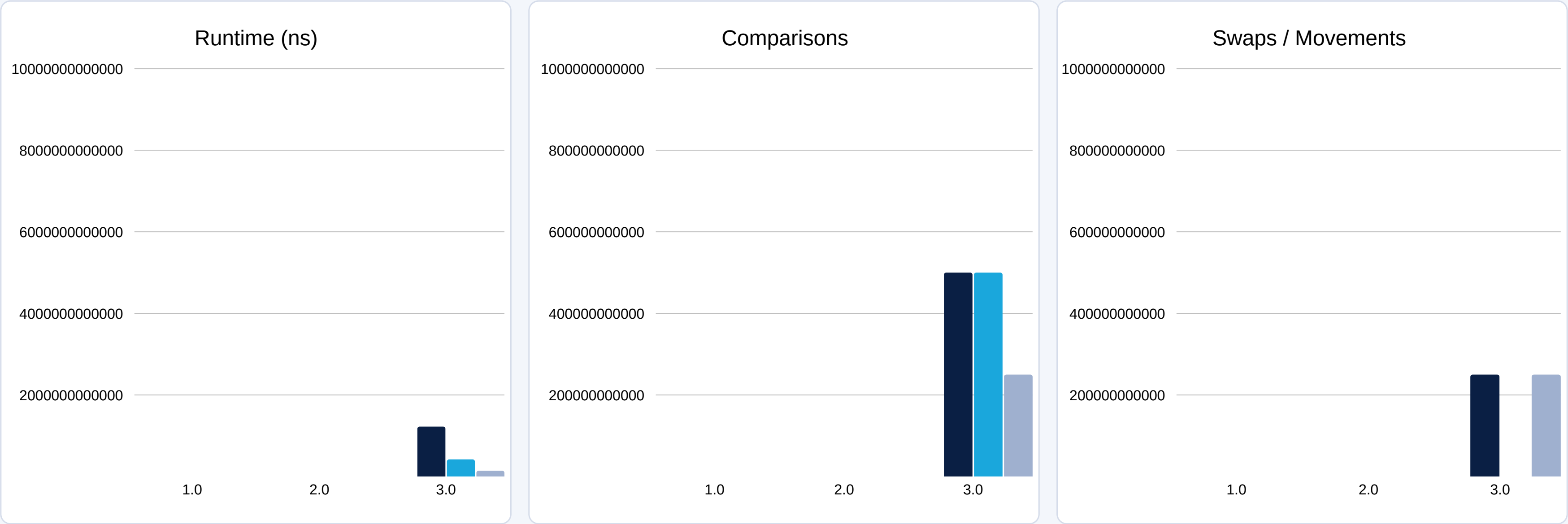
Algorithm	Comparisons	Swaps / Movements	Runtime (ns)	Runtime (s)
Merge Sort	18,673,487	19,951,424	176,428,900	0.18
Bubble Sort	499,995,592,590	250,053,914,853	1,225,725,866,400	1,225.73
Selection Sort	499,999,500,000	999,980	420,229,500,600	420.23
Insertion Sort	250,053,914,853	250,053,914,853	143,563,531,900	143.56



Values from our output file for n = 1,000,000 (single run). Runtime measured in nanoseconds; seconds shown for readability.

Performance Visualization

All three dataset sizes on log-scale charts so small values stay visible



On a log scale, each power of 10 is 10× larger than the previous one. Merge Sort’s bars grow slowly with n , while the $O(n^2)$ sorts grow much faster.

Complexity Comparison

Theory explains the pattern we measured

Algorithm	Best	Average	Worst	Space
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

Why it matters

For $n = 1,000,000$: n^2 is about 10^{12} steps, while $n \log_2 n$ is about 2×10^7 . Our measured Merge Sort comparisons (18.7 million) match this scale.

Merge Sort

$$O(n \log n)$$

Basic sorts: average and worst

$$O(n^2)$$

Doubling n roughly doubles Merge Sort's work, but roughly quadruples the work of $O(n^2)$ sorts.

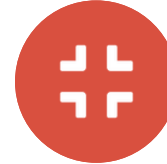
When to Use Merge Sort — and When Not To

Choose the algorithm that fits the data and the constraints

GOOD USE CASES

-  Large datasets
-  Predictable $O(n \log n)$ performance needed
-  Stable sorting is important
-  Sequential-access-style processing
-  External sorting scenarios

LESS APPROPRIATE

-  Very small arrays: a simpler sort has less overhead
-  Memory-constrained situations
-  When creating additional arrays is undesirable

Our results agree: at $n = 5$ Insertion Sort had the lowest runtime; at $n = 1,000$ and $n = 1,000,000$ Merge Sort was fastest.

Real-World Applications and Project Challenges

Where Merge Sort is used, and what we had to handle

REAL-WORLD APPLICATIONS



Database systems



External sorting



Large log files



Records stored on disk



Huge datasets needing predictable performance

PROJECT CHALLENGES

- Implementing recursion and the base case correctly
- Merging two sorted halves correctly
- Tracking comparisons and movements inside merge()
- Testing fairly: same seed, cloned datasets
- Running 1,000,000 elements: Bubble Sort took about 20 minutes
- Measuring consistently: runtimes vary between runs

Conclusion and Member Contributions

Main findings from our Java experiment

- Merge Sort uses divide and conquer: split, sort recursively, merge
- Time is $O(n \log n)$ in best, average, and worst case; space is $O(n)$
- Our Java code uses recursion and a `merge()` that counts comparisons and movements
- $n = 5$: simple sorts were faster because of Merge Sort's overhead
- $n = 1,000$: Merge Sort was about 5–9× faster than the other three
- $n = 1,000,000$: 0.18 s for Merge Sort vs 143.6–1,225.7 s (single run)
- Use measured results together with theoretical complexity

MEMBER CONTRIBUTIONS

Choup Vannphyreak

Implementing setup and testing

Sarou Menglong

Bubble, Selection, and Insertion sort

Taing Tunvatnak

Merge Sort

Doung Piphop

Report analysis and slide preparation

Rin Sovandara

Slide preparation and report

References

Youtube from Bro code

- Merge Sort: <https://youtu.be/3j0SWDX4AtU?si=d93IJ78oPNXy4Plo>
- Bubble Sort: <https://youtu.be/Dv4qLJcxus8?si=D8rqRq1VCK8s8PgA>
- Selection Sort: <https://youtu.be/EwjnF7rFLns?si=n07gMQvVW5Hm0Qu0>
- Insertion Sort: <https://youtu.be/8mJ-OhcfpYg?si=-h211nkS0W7hOV2a>
- Codecademy. (n.d.). Time complexity of merge sort: A detailed analysis.
<https://www.codecademy.com/article/time-complexity-of-merge-sort>
- GeeksforGeeks. (n.d.). Merge sort – Data structure and algorithm tutorials.
<https://www.geeksforgeeks.org/merge-sort/>
- Interview Kickstart. (n.d.). Comparison among selection sort, bubble sort, and insertion sort.
<https://www.interviewkickstart.com/learn/comparison-among-bubble-sort-selection-sort-and-insertion-sort>